

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



BÁO CÁO MÔN CSDL WEB

Đề tài: Quản Lý Chi Tiêu Gia Đình

Giảng viên: TS. Vũ Tiến Dũng

Người hướng dẫn: Phạm Duy Phương

Sinh viên thực hiện: Nguyễn Ngọc Quý - 21002169

Đỗ Đức Vĩnh - 21002181

Đào Tất Thắng - 21002173

MỤC LỤC

1. Giới thiệu.....	6
1.1. Bối cảnh.....	6
1.2. Nhiệm vụ.....	6
1.2.1. Hỗ trợ tạo và quản lý tài khoản người dùng.....	6
1.2.2. Quản lý danh mục chi tiêu.....	6
1.2.3. Ghi nhận và điều chỉnh khoản chi.....	7
1.2.4. Phân tích và báo cáo chi tiêu.....	7
1.2.5. Phân tích và báo cáo chi tiêu.....	7
1.3. Những công cụ và công nghệ được sử dụng.....	7
1.3.1 Ngôn ngữ lập trình:.....	7
1.3.2. Backend.....	7
1.3.3. Frontend.....	9
2. Triển khai.....	11
2.1. Thiết kế sơ lược hệ thống.....	11
2.2. Thiết kế cơ sở dữ liệu quan hệ.....	13
2.5. Triển khai cụ thể.....	14
2.5.1. Khởi tạo kết nối tới MongoDB.....	14
2.5.2. Quản Lý Tài Khoản.....	16
2.5.3. Quản Lý Gia Đình.....	17
2.5.3. Quản Lý Chi Tiêu.....	17
2.5.4. Quản Lý Ngân Sách.....	17
2.5.5. Triển khai Gửi Thông Báo Qua Email.....	18
2.5.6. Triển khai Bảo Mật.....	18
2.5.7. Kiến Trúc Cơ Sở Dữ Liệu.....	18
2.5.8. Triển khai phân quyền.....	18
2.6. Xây dựng API.....	19
2.6.1. API cho người dùng [User].....	20
2.6.2. API cho Admin [Admin].....	21
2.6.3. API cho việc tạo ngân sách và duyệt các yêu cầu thêm ngân sách [Admin].....	22
2.6.4. API cho phần chi phí (User).....	23
2.6.5. API cho báo cáo chi tiêu.....	23
2.7 Xây dựng giao diện.....	24
2.7.1. Hành trình người dùng.....	24
2.7.2. Giao diện người dùng.....	26
4. Giao diện khi chưa có tài khoản.....	35
3. Kết quả đạt được.....	38
3.1. Frontend.....	38
3.2. Backend.....	39

4. Hướng phát triển trong tương lai.....	39
5. Phụ lục.....	40
5.1. Phụ lục 1.....	40
5.2. Phụ lục 2.....	40
6. Phụ lục tham khảo.....	40

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án môn học Cơ Sở Dữ Liệu Web, chúng em đã có nhiều trải nghiệm và bài học quý giá về công việc thực tế sau này. Những kiến thức chúng em học được trong suốt dự án này chắc chắn sẽ mang lại nhiều lợi ích, là nền tảng vững chắc cho sự nghiệp trong tương lai. Nếu không nhận được sự giúp đỡ từ nhiều phía, chắc chắn chúng em sẽ không thể hoàn thành dự án một cách thuận lợi và thành công.

Với sự chỉ đạo và hướng dẫn của anh Phạm Duy Phương - người hướng dẫn môn học, chúng em đã cùng nhau xây dựng lên một sản phẩm mang tính chuyên nghiệp, quy củ và khoa học. Chúng em xin gửi lời cảm ơn chân thành nhất tới anh Phương vì đã trực tiếp hướng dẫn, truyền đạt kiến thức và kinh nghiệm, cũng như dành thời gian quan tâm và hỗ trợ chúng em trong suốt quá trình thực hiện dự án.

Chúng em cũng xin gửi lời cảm ơn tới khoa Toán - Cơ - Tin học, đặc biệt là thầy Vũ Tiến Dũng, đã tạo cơ hội cho chúng em được học tập và làm việc trong môi trường chuyên nghiệp, giúp chúng em hoàn thành đồ án này.

Chúng em xin chân thành cảm ơn!

Nhóm thực hiện

LỜI NÓI ĐẦU

Trong thời đại hiện nay, quản lý chi tiêu gia đình là một nhu cầu thiết yếu nhằm đảm bảo sự ổn định tài chính và cải thiện chất lượng cuộc sống. Đặc biệt, đối với các gia đình nhiều thế hệ, việc theo dõi và kiểm soát chi tiêu của từng thành viên một cách hiệu quả không chỉ giúp tối ưu ngân sách mà còn giảm thiểu các khoản chi không cần thiết.

Website "Quản lý chi tiêu trong gia đình" ra đời nhằm giải quyết những khó khăn này bằng cách cung cấp một nền tảng tiện lợi và trực quan. Dự án cho phép từng thành viên ghi nhận, chỉnh sửa, và phân tích chi tiêu, đồng thời cung cấp các công cụ hữu ích như thiết lập ngân sách, cảnh báo vượt hạn mức, và xuất báo cáo chi tiết. Hệ thống phân quyền linh hoạt đảm bảo tính bảo mật và minh bạch, giúp các thành viên dễ dàng theo dõi tình hình chi tiêu chung.

Với mong muốn hỗ trợ các gia đình quản lý tài chính một cách khoa học, dự án không chỉ hướng đến việc đáp ứng nhu cầu thực tiễn mà còn khuyến khích thói quen quản lý chi tiêu hiệu quả, góp phần tạo nền tảng tài chính vững chắc cho mỗi gia đình.

1. Giới thiệu

1.1. Bối cảnh

- Trong cuộc sống hiện đại, việc quản lý chi tiêu trong gia đình trở nên ngày càng quan trọng để đảm bảo sự cân đối và ổn định tài chính. Một gia đình 3 thế hệ bao gồm Ông Bà, Bố Mẹ và Con thường gặp khó khăn trong việc theo dõi và kiểm soát các khoản chi tiêu hằng ngày, đặc biệt khi mỗi thành viên có những nhu cầu và ưu tiên khác nhau.
- Việc thiếu sự minh bạch và công cụ hỗ trợ có thể dẫn đến những bất đồng hoặc lãng phí tài chính. Vì vậy, một hệ thống quản lý chi tiêu trực tuyến sẽ giúp gia đình dễ dàng ghi nhận, phân tích và kiểm soát các khoản chi, từ đó xây dựng lối sống tiết kiệm, hợp lý hơn.

1.2. Nhiệm vụ

- Mục tiêu của nhóm nghiên cứu chúng em là xây một website quản lý chi tiêu gia đình để giúp người dùng quản lý các khoản chi tiêu hiệu quả hơn. Các nhiệm vụ chính sẽ bao gồm:

1.2.1. Hỗ trợ tạo và quản lý tài khoản người dùng

- Cung cấp chức năng đăng ký, đăng nhập và đăng xuất.
- Phân quyền cho các thành viên trong gia đình theo vai trò. Phân quyền theo chủ gia đình, như các quyền admin và quyền member.
- Cho phép chủ gia đình tạo tài khoản và quản lý quyền hạn, chi tiêu của các thành viên trong gia đình.

1.2.2. Quản lý danh mục chi tiêu

- Tạo các danh mục chi tiêu phổ biến như Ăn uống, Đi lại, Học tập,... và nhiều danh mục khác mà người dùng có thể tự mình thêm vào.
- Ghi nhận thông tin chi tiết của từng khoản chi tiêu, bao gồm số tiền, ngày, và mô tả.

1.2.3. Ghi nhận và điều chỉnh khoản chi

- Hỗ trợ thêm sửa xóa các khoản chi tiêu hằng ngày.

1.2.4. Phân tích và báo cáo chi tiêu

- Cung cấp biểu đồ trực quan để phân tích chi tiêu theo tháng, thành viên, và danh mục.
- Cho phép so sánh mức chi tiêu giữa các thành viên.
- Xuất báo cáo chi tiêu dưới dạng PDF hoặc Excel.

1.2.5. Phân tích và báo cáo chi tiêu

- Cho phép đặt ngân sách hàng tháng cho từng thành viên hoặc danh mục.
- Cảnh báo khi chi tiêu vượt quá ngân sách đã thiết lập.

1.3. Những công cụ và công nghệ được sử dụng.

1.3.1 Ngôn ngữ lập trình:

- Backend: Python - sở hữu hệ sinh thái thư viện phong phú.
- Frontend: Typescript - ngôn ngữ lập trình đang rất được ưa chuộng trong việc xây dựng và thiết kế website, với cộng đồng và thư viện hỗ trợ rộng lớn. Cùng với đó là sự chặt chẽ trong cách xây dựng.

1.3.2. Backend

- *API: FastAPI - framework web hiện đại, được đánh giá cao về tốc độ và hiệu suất.*
- Ưu điểm:
 - + Tốc độ xử lý nhanh chóng nhờ hỗ trợ bất đồng bộ (asynchronous) và sử dụng thư viện Pydantic để kiểm tra kiểu dữ liệu, giúp tăng tốc độ mã hóa và giải mã thông tin.
 - + Dễ sử dụng, cú pháp đơn giản, cho phép tạo API nhanh chóng và dễ dàng.
 - + Tích hợp sẵn Swagger UI, thuận tiện cho việc document và test API.

- + Lựa chọn lý tưởng cho các dự án có quy mô nhỏ, backend và frontend tách biệt, giao tiếp qua API.
- Nhược điểm:
 - + Vẫn còn khá mới mẻ, cộng đồng người dùng và tài liệu chưa phong phú bằng các framework như Django hay Flask.
 - + Hỗ trợ ít tính năng built-in hơn, đòi hỏi phải tự viết thêm code cho các chức năng như authentication, authorization...
- *Hệ quản trị cơ sở dữ liệu: MongoDB - hệ quản trị dữ liệu NOSQL phổ biến*
- Ưu điểm:
 - + Linh hoạt trong việc lưu trữ dữ liệu, sử dụng cấu trúc tài liệu JSON-like (BSON), dễ dàng lưu trữ và làm việc với dữ liệu phi cấu trúc hoặc dữ liệu có cấu trúc không cố định.
 - + Hiệu năng cao: Tối ưu hóa cho các thao tác truy vấn và lưu trữ dữ liệu với khối lượng lớn, phù hợp với các ứng dụng có nhu cầu xử lý nhanh và quy mô lớn.
 - + Hỗ trợ mạnh mẽ các tính năng nâng cao, hỗ trợ kiểu dữ liệu như mảng, nhúng tài liệu, giúp giảm sự phức tạp khi thiết kế cơ sở dữ liệu.
 - + Khả năng mở rộng tốt, dễ dàng mở rộng theo chiều ngang nhờ tính năng sharding.
 - + Cộng đồng lớn và phát triển mạnh, có nhiều tài liệu, công cụ và hỗ trợ từ cộng đồng.
- Nhược điểm:
 - + Thiếu sự ràng buộc mạnh mẽ: MongoDB không áp dụng ràng buộc dữ liệu nghiêm ngặt như các hệ quản trị CSDL quan hệ(SQL).
 - + Dữ liệu lớn hơn do BSON, thường chiếm nhiều dung lượng hơn so với JSON hoặc các hệ quản trị quan hệ.

1.3.3. Frontend

- *Reactjs: Là một thư viện JavaScript mã nguồn mở được phát triển bởi Facebook, dùng để xây dựng giao diện người dùng (UI). Nó cho phép phát triển các ứng dụng web với cấu trúc component, nơi mỗi component có thể quản lý trạng thái và giao diện của riêng mình.*
- Ưu điểm:
 - + Hiệu suất cao
 - + Tái sử dụng Components giúp tổ chức mã nguồn tốt
- Nhược điểm:
 - + Chỉ tập trung vào giao diện
 - + Khó quản lý các trạng thái mới
- *Vite: Là một công cụ build hiện đại cho JavaScript, được thiết kế để tăng tốc quá trình phát triển ứng dụng web. Nó sử dụng ES Modules trong quá trình phát triển và chỉ bundle khi build sản phẩm cuối cùng. Vite thường được sử dụng với các framework như React, Vue, và Svelte.*
- Ưu điểm:
 - + Tốc độ khởi động nhanh nhờ ES Modules
 - + Hỗ trợ module hot-reloading (HMR): Tự động cập nhật giao diện khi chỉnh sửa mã nguồn mà không cần reload lại toàn bộ trang.
 - + Hỗ trợ các plugin và tích hợp tốt với TypeScript, JSX, CSS Modules
- Nhược điểm:
 - + Chỉ hỗ trợ các trình duyệt có hỗ trợ ES Modules
 - + Ít tài liệu hoặc plugin hơn so với Webpack trong một số trường hợp.
- *ShadCN UI: là một thư viện giao diện người dùng dành cho React, tập trung vào việc cung cấp các thành phần thiết kế sẵn dựa trên Radix UI và sử dụng Tailwind CSS để tùy chỉnh. ShadCN UI mang đến sự linh hoạt và hiệu suất*

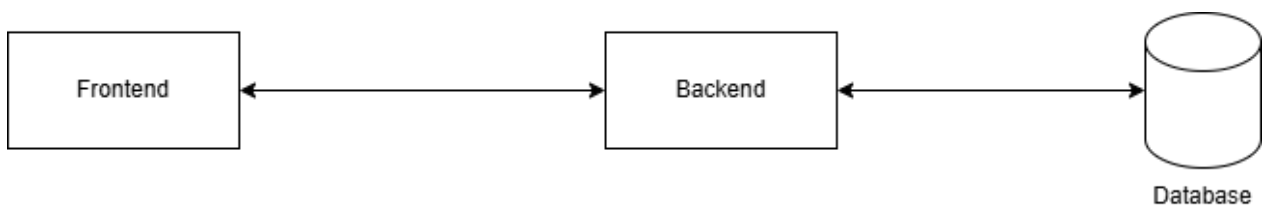
cao, giúp các nhà phát triển tạo ra giao diện hiện đại và có thể mở rộng một cách nhanh chóng mà vẫn giữ được tính nhất quán.

- Ưu điểm:
 - + Kích thước nhỏ gọn, hiệu suất cao, không ảnh hưởng nhiều đến bundle size.
 - + Tùy chỉnh dễ dàng nhờ vào các tiện ích của Tailwind, không cần tạo hệ thống theming phức tạp.
 - + Hỗ trợ các thành phần có thể truy cập và được tối ưu hóa.
 - + Cho phép import các thành phần riêng lẻ, tránh tải thừa không cần thiết.
 - + Phù hợp với các dự án hiện đại sử dụng Tailwind Css.
- Nhược điểm:
 - + Không phù hợp nếu bạn không sử dụng Tailwind trong dự án.
 - + Cộng đồng và tài liệu chưa phong phú như các thư viện lâu đời (Material-UI, Ant Design).
 - + Không cung cấp quá nhiều thành phần sẵn có như các thư viện lớn khác, cần tùy chỉnh thêm.

2. Triển khai

2.1. Thiết kế sơ lược hệ thống

- Hệ thống *Quản lý chi tiêu gia đình* được thiết kế với kiến trúc client-server, nhằm cung cấp giải pháp hiệu quả trong việc theo dõi và quản lý chi tiêu của các thành viên trong gia đình. Hệ thống bao gồm ba thành phần chính: *Frontend*, *Backend* và *Cơ sở dữ liệu*. Phần Frontend, được xây dựng trên các công nghệ như React.js, chịu trách nhiệm cung cấp giao diện người dùng (UI) trực quan, cho phép các thành viên đăng nhập, đăng ký, quản lý danh mục chi tiêu, gửi yêu cầu tăng ngân sách và nhận thông báo. Các yêu cầu từ frontend sẽ được gửi đến Backend thông qua các REST API. Phần Backend được phát triển bằng FastAPI, nơi xử lý toàn bộ logic nghiệp vụ như quản lý người dùng, chi tiêu, ngân sách và thông báo. Đồng thời, backend tích hợp hệ thống email để gửi thông báo trực tiếp đến người dùng thông qua giao thức SMTP. Toàn bộ dữ liệu của hệ thống, bao gồm thông tin người dùng, danh mục chi tiêu, ngân sách và thông báo, được lưu trữ và quản lý trên cơ sở dữ liệu MongoDB.



- Hệ thống được chia thành nhiều module chính, mỗi module đảm nhận một vai trò cụ thể. Module quản lý người dùng cung cấp các chức năng đăng ký, đăng nhập, đăng xuất, phân quyền (admin và thành viên gia đình), cũng như hỗ trợ người dùng cập nhật thông tin cá nhân, quên mật khẩu hoặc thay đổi mật khẩu. Module quản lý gia đình cho phép admin tạo mới một gia đình và thêm các thành viên vào đó, đồng thời hỗ trợ truy vấn thông tin của toàn bộ thành viên trong gia đình. Module quản lý chi tiêu giúp người dùng ghi nhận các khoản chi tiêu hàng ngày, phân loại chi tiêu theo danh mục như ăn uống,

giải trí, đi lại hoặc sức khỏe, và cung cấp chức năng chỉnh sửa hoặc xóa các khoản chi tiêu khi cần. Bên cạnh đó, module quản lý ngân sách cho phép admin thiết lập ngân sách chi tiêu cho từng thành viên hoặc danh mục, tự động cảnh báo khi chi tiêu vượt ngân sách, và hỗ trợ người dùng gửi yêu cầu tăng ngân sách khi cần. Cuối cùng, module quản lý thông báo đảm bảo người dùng luôn được thông tin kịp thời qua email, từ cảnh báo chi tiêu vượt ngân sách đến thông báo về việc phê duyệt hoặc từ chối các yêu cầu tăng ngân sách.

- Quy trình hoạt động của hệ thống được thiết kế mạch lạc, phù hợp với nhu cầu thực tế của các gia đình. Người dùng mới có thể đăng ký tài khoản để trở thành admin, sau đó tạo gia đình và thêm các thành viên khác vào hệ thống. Các thành viên gia đình sau khi được thêm có thể tự do ghi nhận chi tiêu cá nhân của mình và theo dõi tình trạng chi tiêu thông qua các danh mục. Khi phát sinh trường hợp vượt ngân sách, hệ thống sẽ tự động gửi thông báo cho người dùng, đồng thời hỗ trợ họ gửi yêu cầu tăng ngân sách tới admin. Admin sau đó sẽ nhận được thông báo, xem xét và quyết định phê duyệt hoặc từ chối yêu cầu. Hệ thống cũng đảm bảo an toàn và bảo mật thông tin nhờ cơ chế xác thực bằng JSON Web Token (JWT), đồng thời tích hợp hệ thống email để truyền tải thông tin một cách nhanh chóng và hiệu quả.
- Hệ thống được xây dựng trên các công nghệ hiện đại, bao gồm FastAPI cho backend, React.js cho frontend, MongoDB làm cơ sở dữ liệu, và smtplib để gửi thông báo qua email. Kiến trúc hệ thống linh hoạt, dễ mở rộng và tối ưu hóa trải nghiệm người dùng. Với thiết kế này, hệ thống không chỉ đáp ứng nhu cầu quản lý chi tiêu mà còn giúp các thành viên gia đình duy trì tài chính cá nhân và kiểm soát chi tiêu hiệu quả hơn.

2.2. Thiết kế cơ sở dữ liệu quan hệ

- Hệ thống *Family Expense Manager* sử dụng MongoDB làm cơ sở dữ liệu chính, được thiết kế theo cấu trúc NoSQL để tối ưu hóa việc lưu trữ và truy vấn dữ liệu liên quan đến quản lý tài chính gia đình. Cơ sở dữ liệu được chia thành nhiều collection chính, mỗi collection phục vụ một mục đích cụ thể trong ứng dụng.
- Collection *users* lưu trữ thông tin chi tiết về người dùng, bao gồm tên đăng nhập, email, mật khẩu đã băm, vai trò (Admin hoặc thành viên), và ID gia đình mà người dùng thuộc về. Collection này cũng hỗ trợ quản lý phiên làm việc thông qua JWT token và danh sách đen token để tăng cường bảo mật.
- Collection *Token_blacklist* lưu trữ thông tin JWT token mà hệ thống đã thu hồi khi người dùng logout.
- Collection *families* được thiết kế để lưu thông tin về gia đình, chẳng hạn như tên gia đình, ID của Admin, và danh sách các thành viên. Đây là điểm nối giữa các thành viên trong hệ thống, giúp phân định rõ ràng các nhóm sử dụng.
- Để quản lý các khoản chi tiêu, collection *expenses* lưu trữ thông tin chi tiết bao gồm ID danh mục, số tiền chi tiêu, ngày chi tiêu, và mô tả. Mỗi bản ghi trong collection này được liên kết với một người dùng cụ thể thông qua ID, giúp dễ dàng phân biệt và truy vấn theo từng thành viên.
- Collection *categories* chứa các danh mục chi tiêu như Ăn uống, Đi lại, Giải trí,... giúp tổ chức các khoản chi một cách hệ thống và hỗ trợ trong việc phân loại ngân sách.
- Phần quản lý ngân sách được lưu trong collection *budgets*, nơi chứa thông tin về ngân sách được thiết lập cho từng danh mục hoặc thành viên theo từng tháng và năm. Collection này có mối liên kết chặt chẽ với gia đình và danh mục chi tiêu để đảm bảo rằng mọi khoản ngân sách đều được áp dụng chính xác.
- Cuối cùng, hệ thống còn sử dụng collection *budget_requests* để lưu các yêu cầu sửa đổi ngân sách đã gửi, chẳng hạn như yêu cầu phê duyệt ngân sách, giúp Admin dễ dàng quản lý và theo dõi các sự kiện quan trọng.
- Với cấu trúc cơ sở dữ liệu này, hệ thống đảm bảo được tính linh hoạt, dễ mở rộng, và khả năng xử lý dữ liệu lớn, phù hợp với mục tiêu quản lý tài chính

của các gia đình trong ứng dụng. MongoDB cung cấp khả năng truy vấn nhanh chóng, phù hợp với các yêu cầu truy xuất thông tin chi tiết và thống kê tài chính trong thời gian thực.

2.5. Triển khai cụ thể

2.5.1. Khởi tạo kết nối tới MongoDB

- Khi làm việc với cơ sở dữ liệu MongoDB trong Python, chúng em đã chọn sử dụng thư viện *motor*, một driver MongoDB không đồng bộ dành cho Python. Thư viện này được tải và cài đặt trước khi tạo một đối tượng kết nối với MongoDB trong một module Python, cho phép các module khác dễ dàng truy cập cơ sở dữ liệu bằng cách nhập đối tượng kết nối từ module đó.
- Đối tượng kết nối được khởi tạo bằng cách sử dụng [AsyncIOMotorClient](#), kết nối với MongoDB thông qua URI được cấu hình sẵn trong ứng dụng. Thông qua đối tượng này, chúng em đã thiết lập các collection như [users_collection](#), [families_collection](#), hay [expenses_collection](#) để tương tác với các bảng dữ liệu tương ứng trong MongoDB. Cách làm này không chỉ giúp tối ưu hóa hiệu suất truy vấn mà còn cung cấp sự thuận tiện trong việc quản lý và thao tác dữ liệu. Thư viện **motor** hỗ trợ tốt các thao tác bất đồng bộ, giúp ứng dụng hoạt động mượt mà hơn trong các tác vụ cần nhiều I/O như làm việc với cơ sở dữ liệu.

```

1 from motor.motor_asyncio import AsyncIOMotorClient
2 from app.core.config import settings
3
4 client = AsyncIOMotorClient(settings.MONGODB_URI)
5 db = client[settings.MONGODB_DB_NAME]
6
7 users_collection = db.get_collection("users")
8 families_collection = db.get_collection("families")
9 expense_categories_collection = db.get_collection(
10     "expense_categories")
11 expenses_collection = db.get_collection("expenses")
12 budgets_collection = db.get_collection("budgets")
13 notifications_collection = db.get_collection("notifications")
14 budget_requests_collection = db.get_collection(
15     "budget_requests")
16 token_blacklist_collection = db.get_collection(
17     "token_blacklist")

```

- Trong module cấu hình, một lớp `Settings` được định nghĩa, kế thừa từ `BaseSettings` của Pydantic. Lớp này giúp ứng dụng dễ dàng quản lý các giá trị cấu hình thông qua việc ánh xạ trực tiếp với các biến môi trường hoặc các giá trị mặc định được cung cấp. Các tham số cấu hình quan trọng như `MONGODB_URI`, `MONGODB_DB_NAME`, và `SECRET_KEY` được khai báo rõ ràng với kiểu dữ liệu cụ thể. Điều này không chỉ đảm bảo tính chính xác của dữ liệu mà còn hỗ trợ tốt trong việc phát hiện lỗi. Ngoài ra, lớp cấu hình còn quản lý thông tin liên quan đến hệ thống email (`EMAIL_SERVER`, `EMAIL_PORT`, `EMAIL_USER`, `EMAIL_PASSWORD`), cùng với các thông số khác như thời gian hết hạn của token truy cập (`ACCESS_TOKEN_EXPIRE_MINUTES`).
- Nhờ cơ chế tự động tải giá trị từ tệp môi trường (`.env`), ứng dụng có thể dễ dàng điều chỉnh cấu hình mà không cần thay đổi mã nguồn. Điều này giúp đơn giản hóa việc quản lý cấu hình, tăng cường bảo mật và linh hoạt trong các môi trường triển khai khác nhau.

```
1 # app/core/config.py
2 from pydantic_settings import BaseSettings
3 from pydantic import Field
4
5 class Settings(BaseSettings):
6     MONGODB_URI: str = Field(default=
7     "mongodb://localhost:27017")
8     MONGODB_DB_NAME: str = Field(default="family_expense_db")
9     SECRET_KEY: str = Field(..., env="SECRET_KEY")
10    ACCESS_TOKEN_EXPIRE_MINUTES: int = Field(default=60 * 24)
11    EMAIL_SERVER: str = Field(..., env="EMAIL_SERVER")
12    EMAIL_PORT: int = Field(..., env="EMAIL_PORT")
13    EMAIL_USER: str = Field(..., env="EMAIL_USER")
14    EMAIL_PASSWORD: str = Field(..., env="EMAIL_PASSWORD")
15
16    class Config:
17        env_file = ".env"
18
19 settings = Settings()
```

2.5.2. Quản Lý Tài Khoản

Ứng dụng hỗ trợ các chức năng quản lý tài khoản cơ bản, bao gồm:

- *Đăng ký và đăng nhập*: Người dùng có thể đăng ký tài khoản với vai trò là Admin. Admin có quyền tạo và quản lý gia đình với trường hợp admin đó chưa tạo gia đình trong ứng dụng bao giờ, admin có quyền tạo tài khoản cho các thành viên trong gia đình. Trong phần đăng ký, mật khẩu của người dùng được băm nhỏ bởi hàm `hash_password(password: str)` để nâng cao tính bảo mật trước khi lưu vào CSDL
- *Đăng xuất*: Đảm bảo bảo mật phiên làm việc bằng cách ngắt kết nối token khi người dùng thoát ứng dụng.
- *Quên mật khẩu*: Người dùng có thể nhận được mật khẩu mới (đã băm) qua email trong trường hợp quên mật khẩu.

2.5.3. Quản Lý Gia Đình

Chức năng này giúp Admin dễ dàng quản lý các thành viên trong gia đình:

- *Tạo gia đình*: Admin được yêu cầu tạo gia đình sau khi đăng ký.
- *Quản lý thành viên*: Admin có thể thêm thành viên vào gia đình.
- *Xem danh sách thành viên*: Người dùng có thể xem thông tin chi tiết của tất cả các thành viên trong gia đình.

2.5.3. Quản Lý Chi Tiêu

Chức năng quản lý chi tiêu được xây dựng để người dùng theo dõi và kiểm soát các khoản chi:

- *Tạo khoản chi tiêu*: Người dùng có thể tạo các khoản chi với thông tin danh mục, số tiền, ngày và mô tả.
- *Chỉnh sửa hoặc xóa khoản chi*: Đảm bảo khả năng người dùng có thể sửa đổi hoặc xóa các khoản chi trong trường hợp có sai sót. Ngoài ra tính toán khoản chi mà người dùng tạo có vượt quá mức ngân sách hay không, nếu vượt qua thì hệ thống sẽ gửi email thông báo đến admin và người dùng, sau đó hệ thống sẽ đề xuất cho người dùng có thể xin thêm ngân sách bằng cách gửi email đến admin. Admin có thể cho phép nâng mức ngân sách ấy hoặc từ chối nếu không thấy hợp lý.
- *Hiển thị danh sách chi tiêu*: Danh sách hiển thị đầy đủ thông tin như tên danh mục, tên người thực hiện, số tiền và ngày đảm bảo cho người dùng có thể kiểm soát được chi tiết các khoản chi.

2.5.4. Quản Lý Ngân Sách

Tính năng quản lý ngân sách cho phép Admin kiểm soát tài chính của gia đình:

- *Thiết lập ngân sách*: Admin có thể đặt ngân sách cho từng danh mục hoặc từng thành viên, áp dụng theo tháng và năm. Tính năng này chỉ có admin mới có thể thiết lập, vì vậy cần phân quyền qua hàm `check_role(required_role: str)` để xác định được vai trò của người dùng.
- *Chỉnh sửa và xóa ngân sách*: Admin có thể điều chỉnh hoặc xóa ngân sách khi cần.
- *Kiểm tra vượt ngân sách*: Ứng dụng tự động kiểm tra tổng chi tiêu của danh mục hoặc thành viên, gửi thông báo khi vượt quá ngân sách qua email.

2.5.5. Triển khai Gửi Thông Báo Qua Email

Khi xảy ra các sự kiện quan trọng, hệ thống sẽ tự động gửi email thông báo:

- *Thông báo vượt ngân sách:* Nếu tổng chi tiêu vượt ngân sách, email sẽ được gửi đến người dùng và Admin.
- *Yêu cầu phê duyệt ngân sách:* Thành viên có thể gửi yêu cầu tăng ngân sách đến Admin, và Admin sẽ nhận được thông báo qua email để phê duyệt hoặc từ chối.

2.5.6. Triển khai Bảo Mật

Ứng dụng tích hợp các cơ chế bảo mật để đảm bảo an toàn dữ liệu người dùng:

- *Sử dụng JWT token:* Quản lý phiên làm việc thông qua token với danh sách đen để ngăn chặn việc sử dụng lại token bị thu hồi.
- *Mã hóa mật khẩu:* Tất cả mật khẩu người dùng được băm trước khi lưu trữ.
- *Phân quyền truy cập:* Đảm bảo chỉ người dùng có quyền hạn mới được phép truy cập các API tương ứng.

2.5.7. Kiến Trúc Cơ Sở Dữ Liệu

Cơ sở dữ liệu được thiết kế với các bảng chính:

- *Người dùng:* Lưu trữ thông tin tài khoản, vai trò và quyền hạn.
- *Gia đình:* Quản lý thông tin gia đình và các thành viên.
- *Chi tiêu:* Theo dõi thông tin chi tiêu chi tiết.
- *Ngân sách:* Lưu thông tin ngân sách cho từng danh mục hoặc thành viên.
- *Danh mục chi tiêu:* Quản lý các danh mục như Ăn uống, Đi lại, Giải trí,...
- *Token Black list:* Quản lý các token đã thu hồi
- *Yêu cầu ngân sách:* Quản lý danh sách các yêu cầu tăng ngân sách

2.5.8. Triển khai phân quyền

Dự án này tích hợp chức năng phân quyền giữa *Admin* (Chủ gia đình) và *Member* (Thành viên trong gia đình), đảm bảo rằng quyền hạn và thông tin được quản lý phù hợp với vai trò của từng người dùng.

- Phân quyền chi tiết
 - *Admin*
 - Xem thông tin chi tiết của tất cả các thành viên trong gia đình.
 - Tạo tài khoản mới cho các thành viên trong gia đình
 - Tạo trực tiếp ngân sách cá nhân (Budget) cho từng thành viên mà không cần sự phê duyệt hoặc yêu cầu từ bất kỳ ai.
 - Theo dõi, cập nhật và xóa ngân sách của từng thành viên.
 - Ngân sách cá nhân của từng thành viên.
 - Lịch sử yêu cầu ngân sách từ các thành viên.
 - *Member*
 - Chỉ được phép xem thông tin cá nhân của chính mình và 1 số thông tin chung của gia đình
 - Không được tự tạo ngân sách cho bản thân.
 - Khi cần ngân sách, Member phải tạo yêu cầu (Request Budget) gửi tới Admin thông qua hệ thống. Yêu cầu sẽ được chuyển tới email của Admin hoặc hiển thị trong hàng chờ (Queue) để Admin duyệt.
 - Sau khi Admin xem xét, Member sẽ nhận thông báo kết quả qua hệ thống hoặc email (chấp nhận hoặc từ chối).

2.6. Xây dựng API

- Trong một dự án có sự phân chia rõ ràng giữa BE và FE, việc xây dựng API trở nên hết sức quan trọng. API cho phép hai bên giao tiếp hiệu quả hơn bằng cách tuân theo một quy ước chung. Do đó, trong quá trình phát triển BE, chúng em cũng phát triển song song các API, giúp bên FE có thể sử dụng và kiểm tra tính năng của code BE trên giao diện FE. Mọi API đều trả về dữ liệu dạng JSON, giúp việc làm việc giữa BE và FE trở nên thuận lợi hơn. Sau đây là danh sách các API endpoints mà chúng tôi đã phát triển và áp dụng trong dự án.

2.6.1. API cho người dùng [User]

Authentication



POST

/auth/register Register



POST

/auth/login Login



POST

/auth/logout Logout



POST

/auth/forget-password Forget Password



PUT

/auth/change-password Change Password



2.6.2. API cho Admin [Admin]

Family

**POST****/family/create** Create Family

User Management

**POST****/users/add_members** Add Members**GET****/users/family** Get All Users In Family

Expense Categories

**GET****/expense-categories/** Get Expense Categories**POST****/expense-categories/** Create Expense Category**PUT****/expense-categories/{category_id}** Update Expense Category**DELETE****/expense-categories/{category_id}** Delete Expense Category

2.6.3. API cho việc tạo ngân sách và duyệt các yêu cầu thêm ngân sách [Admin]

Budgets



GET	/budgets/	Get Budgets		
POST	/budgets/	Create Budget		
GET	/budgets/{budget_id}	Get Budget		
PUT	/budgets/{budget_id}	Update Budget		
DELETE	/budgets/{budget_id}	Delete Budget		

Budget Requests



POST	/budget-requests/	Create Budget Request		
GET	/budget-requests/pending	Get Pending Requests		
PUT	/budget-requests/approve/{request_id}	Approve Budget Request		
PUT	/budget-requests/deny/{request_id}	Deny Budget Request		

2.6.4. API cho phần chi phí (User)

Expenses



GET	/expenses/	Get Expenses		
POST	/expenses/	Create Expense		
PUT	/expenses/{expense_id}	Update Expense		
DELETE	/expenses/{expense_id}	Delete Expense		

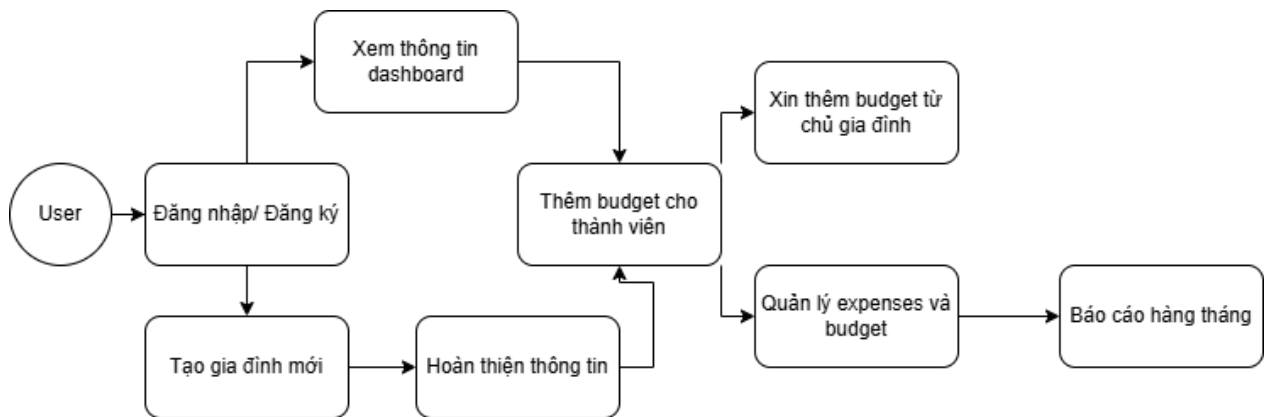
2.6.5. API cho báo cáo chi tiêu

GET	/expenses/family-data	Get Family Data		
GET	/expenses/member-data	Get Member Data		
GET	/expenses/monthly-data	Get Monthly Data		

2.7 Xây dựng giao diện

2.7.1. Hành trình người dùng

- Sơ đồ hành trình cơ bản của người dùng khi sử dụng Family Expenses Management



1. Đăng nhập/ đăng ký

- Người dùng mới bắt đầu bằng việc đăng ký tài khoản trong hệ thống, cung cấp thông tin cá nhân và xác thực tài khoản qua email.
- Người dùng đã có tài khoản có thể đăng nhập trực tiếp để truy cập vào hệ thống quản lý chi phí gia đình.

2. Dashboard

- Sau khi đăng nhập thành công, người dùng sẽ được đưa đến **Dashboard** — nơi hiển thị tổng quan về tình hình tài chính của gia đình, bao gồm tổng ngân sách, chi tiêu hàng tháng, và các khoản còn lại.

3. Tạo gia đình mới

- Nếu người dùng chưa là thành viên của gia đình nào, họ có thể tạo một gia đình mới bằng cách thêm tên gia đình và mời các thành viên tham gia.
- Quá trình này bao gồm việc chỉ định người đứng đầu gia đình (Chủ gia đình) tạo tài khoản cho các thành viên khác

4. Hoàn thiện thông tin gia đình

- Người dùng và các thành viên gia đình cần cung cấp thêm thông tin cá nhân, bao gồm các khoản ngân sách đã có, mục tiêu tài chính gia đình, và các chi tiêu cố định hàng tháng.
- Thông tin này sẽ giúp hệ thống tính toán và phân bổ ngân sách hợp lý.

5. Thêm Budget (Ngân sách) cho các thành viên

- Chủ gia đình có thể tạo và phân bổ ngân sách cho từng thành viên trong gia đình.
- Các thành viên có thể nhận ngân sách riêng và theo dõi việc chi tiêu cá nhân.

6. Xin Budget từ Chủ gia đình:

- Nếu một thành viên cần thêm ngân sách cho các khoản chi tiêu khẩn cấp, họ có thể gửi yêu cầu xin ngân sách từ Chủ gia đình.
- Chủ gia đình sẽ xem xét và phê duyệt hoặc từ chối yêu cầu đó dựa trên tình hình tài chính của gia đình

7. Quản lý Budget và Expenses (Ngân sách và Chi tiêu)

- Người dùng có thể theo dõi ngân sách của mình, xem các khoản chi tiêu hàng tháng, và cập nhật các khoản chi mới.
- Các thành viên gia đình có thể báo cáo chi tiêu của mình và xem lại lịch sử chi tiêu, giúp mọi người quản lý tài chính tốt hơn.

8. Báo cáo hàng tháng:

- Hệ thống cung cấp báo cáo tài chính hàng tháng, giúp người dùng và Chủ gia đình nắm rõ tình hình chi tiêu so với ngân sách đã phân bổ.
- Báo cáo này sẽ có chi tiết về các khoản chi tiêu, tiết kiệm, cũng như các dự báo tài chính trong tương lai.

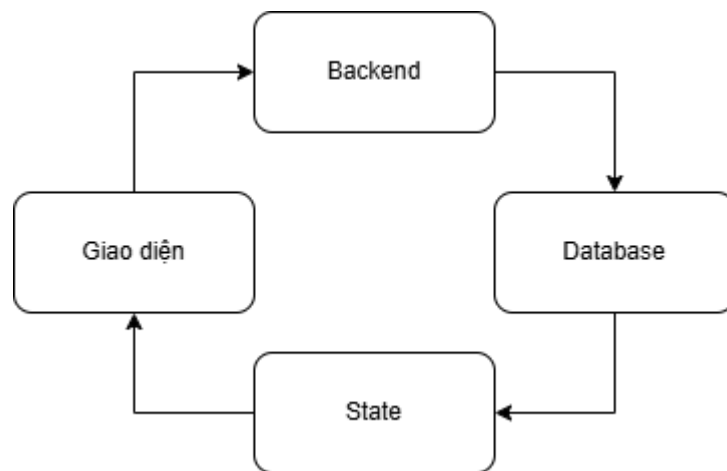
2.7.2. Giao diện người dùng

1. Mục tiêu và yêu cầu giao diện

- Mục tiêu chính:

- + Giao diện thân thiện, dễ sử dụng
- + Đảm bảo trực quan tốt hiển thị các thông tin chi phí trong gia đình
- + Nâng cao hiệu suất trang web giúp trải nghiệm người dùng mượt mà
- + Tương thích tốt với nhiều thiết bị khác nhau (máy tính - máy tính bảng)

2. Sơ đồ hoạt động

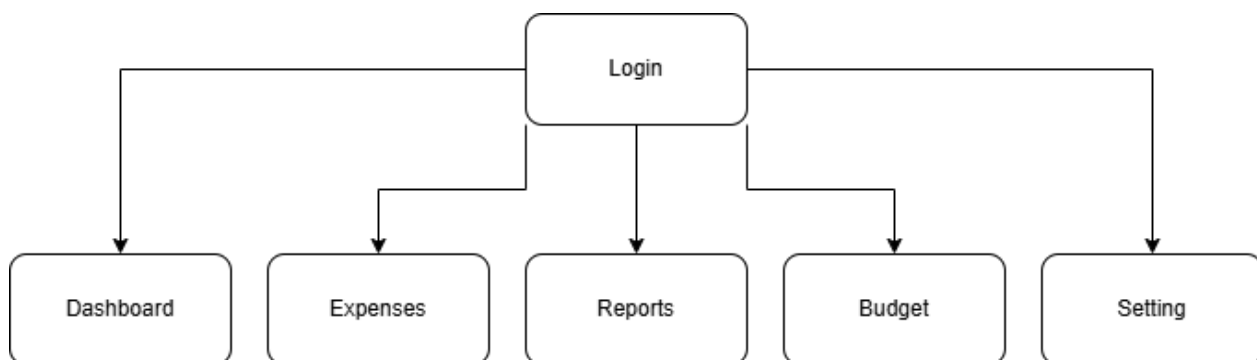


- Sơ đồ trên thể hiện cách hoạt động bao quát của FrontEnd:

- + React sẽ gửi các API đến backend. Sau khi đợi backend lấy thông tin và truy xuất từ CSDL, dữ liệu sẽ được truyền tới react và lưu vào trong state.
- + Sau khi đã có thông tin từ server và lưu vào state, nó sẽ được render với giao diện người dùng

3. Giao diện khi đã có tài khoản

- Sơ đồ giao diện **khi đã có tài khoản** và các thành phần giao diện của trang web được thiết kế như sau:



Login

Enter your email and password to login to your account

Email

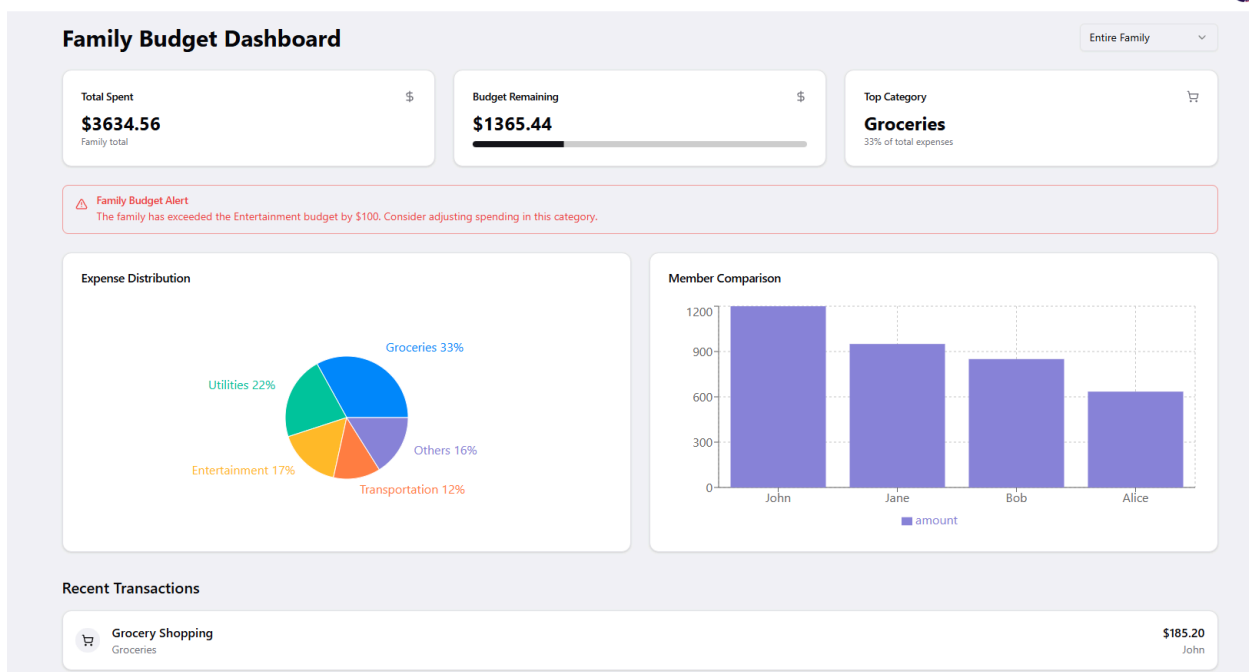
m@example.com

Password

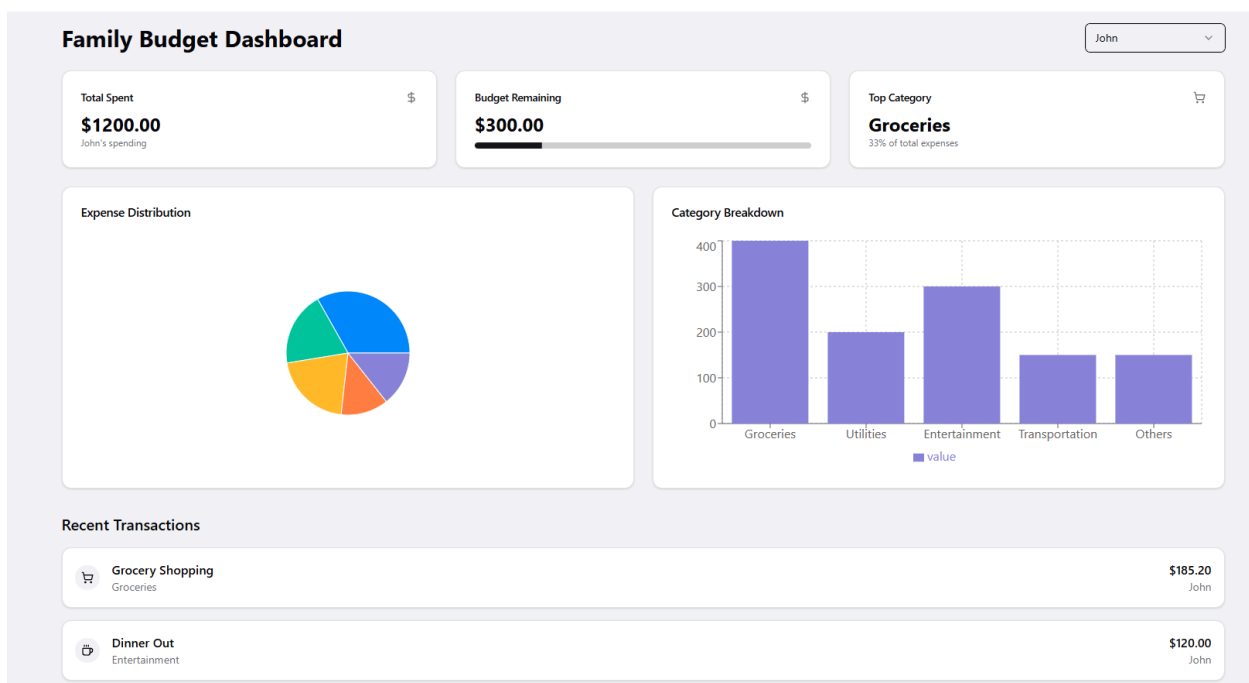
→] Login

Don't have an account? [Register](#)

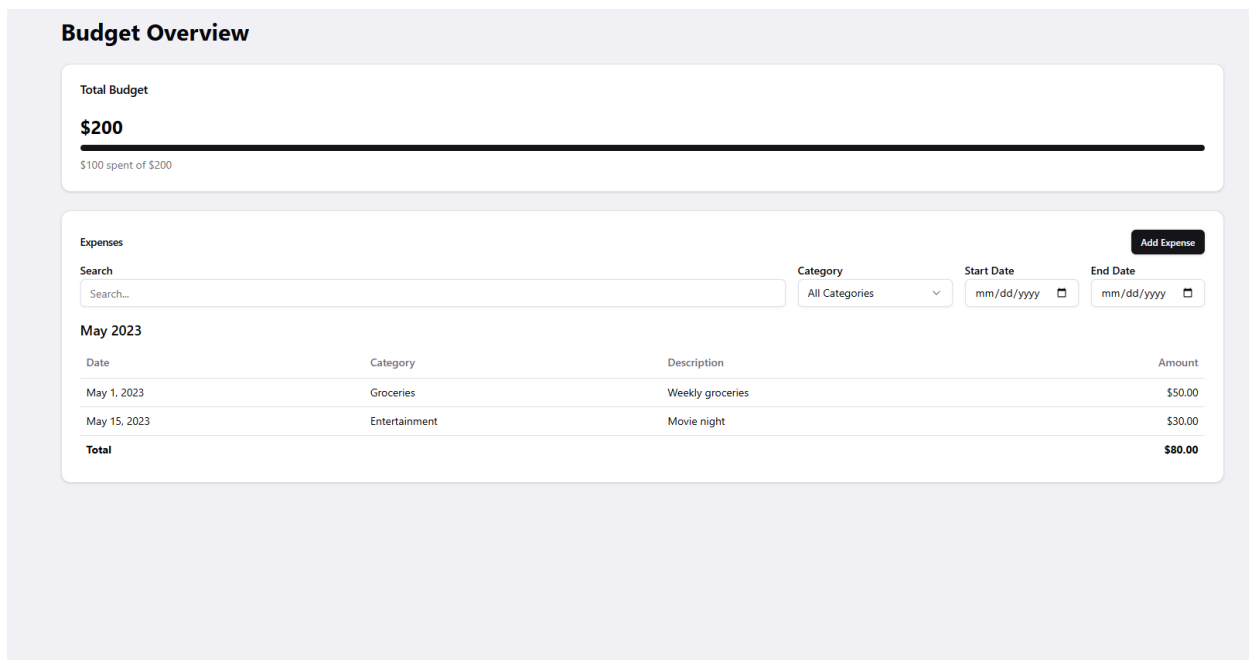
***Login:** Đăng nhập nếu đã có tài khoản*



Dashboard: Giao diện dashboard hiển thị thông tin tổng quan về những chi phí trong gia đình



Dashboard: Ngoài ra giao diện dashboard có thể xem được báo cáo của từng thành viên



***Budget Page:** Xem thông tin budget theo ngày tháng hoặc category*

Set New Budget

Amount

Category Select category

Period

Set Budget

***Set New Budget:** Có thể tạo thêm budget cho gia đình, hoặc member có thể xin thêm*

Expenses Overview

Total Expenses

\$200

\$100 spent of \$200

Expenses

Add Expense

Search

Search...

Category

All Categories

Start Date

mm/dd/yyyy

End Date

mm/dd/yyyy

May 2023

Date	Category	Description	Amount
May 1, 2023	Groceries	Weekly groceries	\$50.00
May 15, 2023	Entertainment	Movie night	\$30.00
Total			\$80.00

Expenses Page: Xem thông tin và lịch sử chi phí trong gia đình

Add New Expense

Amount

Category

Select category

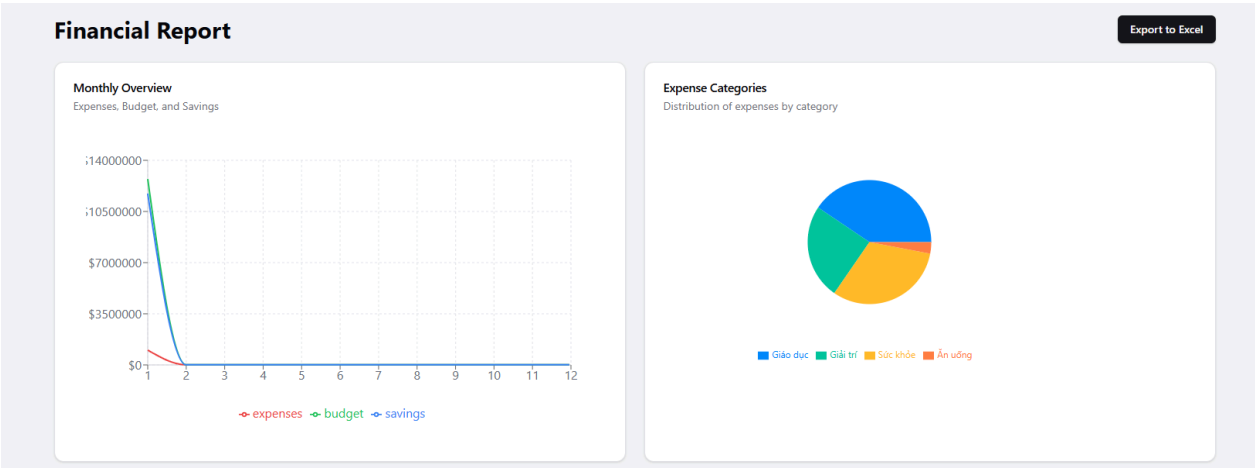
Date

mm/dd/yyyy

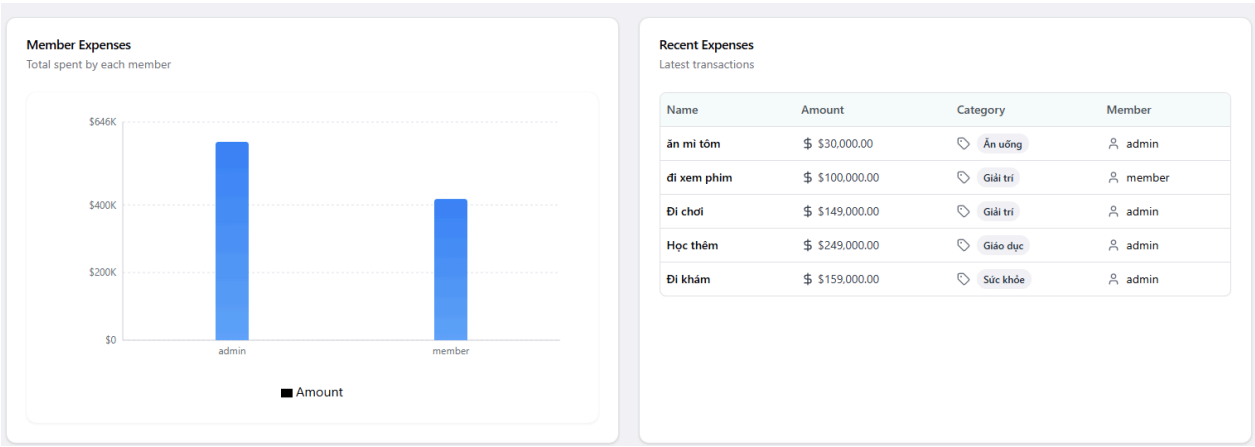
Description

Add Expense

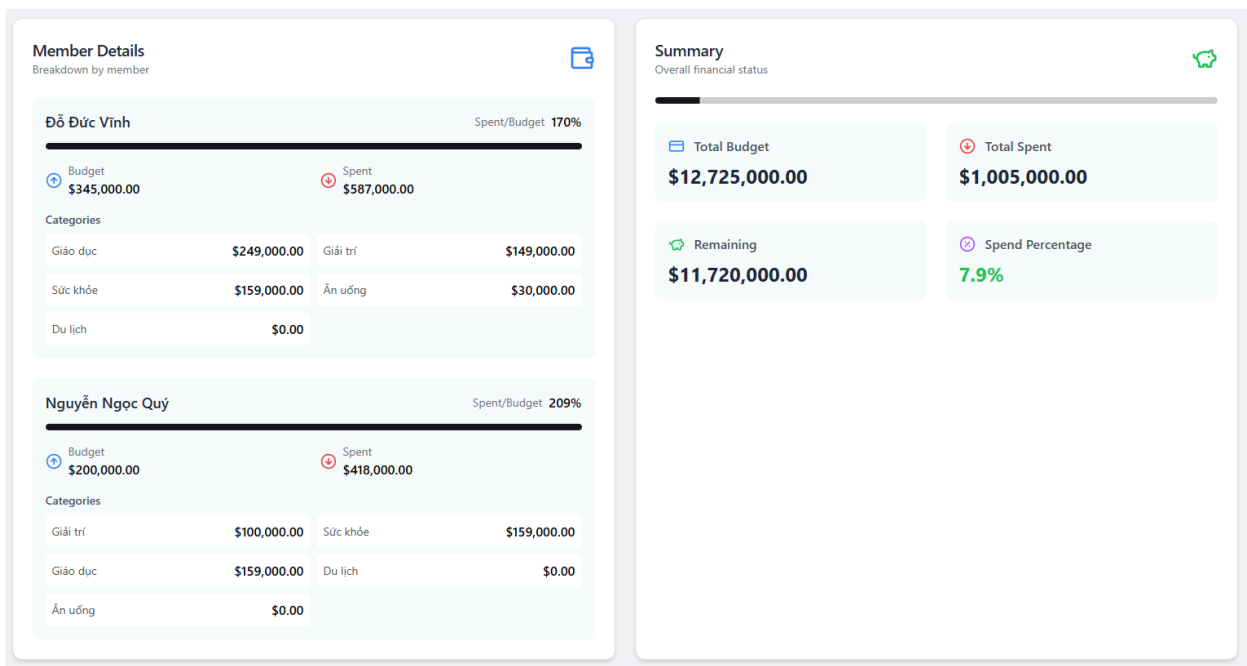
Add Expenses: Thêm thông tin về chi phí mới cho một category



Reports: Báo cáo chi tiết về các khoản chi tiêu trong gia đình



Reports: Báo cáo chi tiết về các khoản chi tiêu trong gia đình



Reports: Báo cáo chi tiết về các khoản chi tiêu trong gia đình

Cài đặt thành viên gia đình

Thêm thành viên

Danh sách thành viên

Họ tên	Tên đăng nhập	Vai trò	Email	Vai trò cụ thể	Thao tác
Đỗ Đức Vinh	admin	admin			
Nguyễn Ngọc Quý	member	member			

Danh sách danh mục

Thêm danh mục

Tên danh mục	ID Gia đình	Thao tác
Giáo dục	678a19f9a9c610df70ad5388	
Giải trí	678a19f9a9c610df70ad5388	
Sức khỏe	678a19f9a9c610df70ad5388	
Du lịch	678a19f9a9c610df70ad5388	
Ăn uống	678a19f9a9c610df70ad5388	

Setting Page: Xem thông tin tài khoản của từng thành viên trong gia đình

Add New Family Member

×

Full Name

Username

Email

Password

Confirm Password

Role

Select a role

Specific Role

Select a specific role

Add Member

Setting Page: Tạo tài khoản mới cho một thành viên mới

- Tải xuống báo cáo hàng tháng từ dữ liệu trong database, dữ liệu tải về sẽ có những định dạng và bảng như sau:

month	expenses	budget	savings
1	1005000	12725000	11720000
2	0	0	0
3	0	0	0
4	0	0	0
5	0	0	0
6	0	0	0
7	0	0	0
8	0	0	0
9	0	0	0
10	0	0	0
11	0	0	0
12	0	0	0

Bảng 1: Tổng Chi phí, chi tiêu, tiết kiệm hàng tháng

Category	Amount	Member	Amount
Giáo dục	408000	admin	587000
Giải trí	249000	member	418000
Sức khỏe	318000		
Ăn uống	30000		

Bảng 2: Chi phí theo từng thành viên và từng danh mục

Name	Amount	Category	Member
ăn mì tôm	30000	Ăn uống	admin
đi xem phim	100000	Giải trí	member
Đi chơi	149000	Giải trí	admin
Học thêm	249000	Giáo dục	admin
Đi khám	159000	Sức khỏe	admin

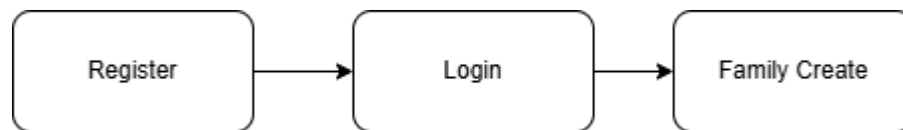
Bảng 3: Lịch sử chi tiêu

Member	Category	Amount	Total Budget	Total Spent	Remaining	Spend %
Đỗ Đức Vĩnh	Giáo dục	249000	345000	587000	-242000	170.14
Đỗ Đức Vĩnh	Giải trí	149000	345000	587000	-242000	170.14
Đỗ Đức Vĩnh	Sức khỏe	159000	345000	587000	-242000	170.14
Đỗ Đức Vĩnh	Ăn uống	30000	345000	587000	-242000	170.14
Đỗ Đức Vĩnh	Du lịch	0	345000	587000	-242000	170.14
Nguyễn Ngọc Quý	Giải trí	100000	200000	418000	-218000	209
Nguyễn Ngọc Quý	Sức khỏe	159000	200000	418000	-218000	209
Nguyễn Ngọc Quý	Giáo dục	159000	200000	418000	-218000	209
Nguyễn Ngọc Quý	Du lịch	0	200000	418000	-218000	209
Nguyễn Ngọc Quý	Ăn uống	0	200000	418000	-218000	209

Bảng 4: Báo cáo chi tiết của từng thành viên

4. Giao diện khi chưa có tài khoản

- Sơ đồ giao diện **khi chưa có tài khoản** và các thành phần giao diện của trang web được thiết kế như sau:



Create an account

Enter your information to create your account

Full Name

John Doe

Email

m@example.com

Username

example

Password

Confirm Password

Roles

Select role

Register

Already have an account?

Login

Register: Tạo tài khoản mới cho gia đình mới, đây sẽ là tài khoản admin

Family Information Required

×

Before proceeding, you need to set up your family information. This will help us personalize your experience and organize your household finances better.

Setting up your family profile includes:

- Creating a family name
- Setting up expense categories for better organization

Create Family Profile

Require: Yêu cầu tạo gia đình trước khi bắt đầu làm việc

Create Family - Step 1

Follow the steps to create your family

Step 1 of 250% Complete

Family Name

Enter your family name

Next

Create Family - Step 1: Đặt tên cho gia đình

Create Family - Step 2

Follow the steps to create your family

Step 2 of 2

100% Complete

Expense Categories

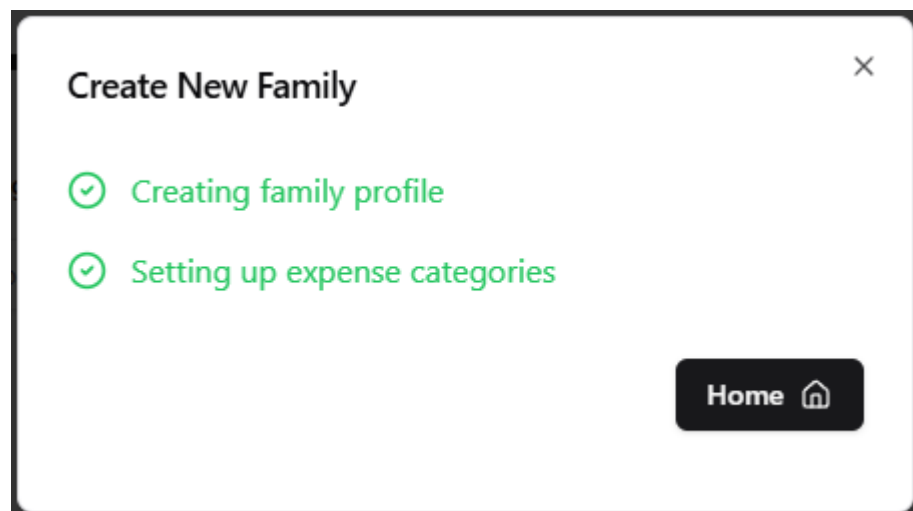
New category

Add

Previous

Finish

Create Family - Step 1: Tạo các category cho các chi tiêu sau này



Tạo thông tin gia đình thành công

Các giao diện sau sẽ tương tự với khi đã có tài khoản

3. Kết quả đạt được

3.1. Frontend

- Thiết kế giao diện trực quan, dễ sử dụng với các thiết kế dễ nhìn, hiện đại, tập trung vào trải nghiệm người dùng (UX).

- Tốc độ tải trang và phản hồi của giao diện đã được tối ưu, đảm bảo trải nghiệm người dùng mượt mà ngay cả khi với khối dữ liệu khá lớn.
- Giao diện hiển thị tốt các chức năng thêm, xóa, sửa các budget, expenses và các thành viên trong gia đình

3.2. Backend

- ***Cơ sở dữ liệu***

- Hệ thống backend hoàn thiện và tối ưu với cơ sở dữ liệu, đảm bảo việc truy xuất và lưu trữ thông tin, quản lý dữ liệu cá nhân người dùng một cách an toàn và nhanh chóng
- Thiết kế cơ sở dữ liệu hệ thống và hệ cơ sở dữ liệu vector lưu trữ văn bản và các chunk của văn bản linh hoạt

- ***API***

- Xây dựng API theo FastAPI mạnh mẽ, hỗ trợ đầy đủ các thao tác CRUD (Create, Read, Update, Delete) cho tài liệu và thông tin người dùng.
- Tối ưu hiệu suất các API, giảm thiểu độ trễ và tăng khả năng đáp ứng nhiều request đến từ người dùng.
- Cấu hình CORS đảm bảo bảo mật cho các API, bao gồm xác thực người dùng và mã hóa dữ liệu.

- ***Quản lý phiên và người dùng***

- Hệ thống xác thực và phân quyền người dùng được xây dựng và kiểm tra kỹ lưỡng, đảm bảo chỉ người dùng hợp lệ mới truy cập được thông tin của họ, ngoài ra chỉ có các tài khoản admin được sử dụng để quản lý trang web.
- Backend hỗ trợ quản lý phiên làm việc người dùng một cách an toàn và hiệu quả, đảm bảo dữ liệu luôn được lưu trữ và truy cập chính xác.

4. Hướng phát triển trong tương lai

- Tích hợp AI: Dự đoán chi tiêu, gợi ý tiết kiệm và tối ưu hóa ngân sách.
- Phân tích chi tiêu nâng cao: Báo cáo chi tiết, biểu đồ trực quan theo thời gian thực.

- Nhắc nhở và cảnh báo: Tự động nhắc các khoản chi quan trọng hoặc cảnh báo chi tiêu vượt hạn mức.
- Tích hợp thanh toán: Liên kết với ví điện tử, ngân hàng để tự động ghi nhận giao dịch.
- Tùy chỉnh theo người dùng: Tính năng lập mục tiêu tài chính và kế hoạch phù hợp cho từng gia đình.

5. Phụ lục

5.1. Phụ lục 1

Tài liệu hướng dẫn sử dụng chi tiết: [Link](#)

Hướng dẫn cài đặt giao diện người dùng: [Link](#)

Tài liệu hướng dẫn cài đặt Backend: [Link](#)

5.2. Phụ lục 2

Tài liệu thiết kế giao diện chi tiết: [Link](#)

6. Phụ lục tham khảo

Tài liệu API: [Link](#)

FastApi: Hướng dẫn xây dựng API: [FastAPI Documentation](#)

React: Tài liệu phát triển giao diện người dùng: [React Documentation](#)

ShadcnUI: Thư viện giao diện React: [Shadcn UI Documentation](#)

TailwindCss: Framework Css: [Tailwind CSS Documentation](#)

MimeMultipart: Xử lý dữ liệu MIME: [MimeMultipart Documentation](#)